

FULL STACK DEVELOPER TECHNICAL TEST

ROI Advertising Calculator

(AdForecast Pro)

Production-ready real-time campaign calculator, user data isolation, and deployment

Candidate: **Alvian Rahmadani**

Tech Stack: **Laravel 12 (PHP 8.4) + React 19 + Inertia v3 + PostgreSQL**

Requirements & Business Scope

Secure Authentication

Hashed passwords, session/cookie authentication that natively persists on page refresh, and strictly protected calculator & history routes.

Real-Time ROI Calculator

Instant client-side calculation updating Revenue, Margin, ROI, and Target CPR simultaneously with 2-way synchronized slider & number inputs.

Private User Data Isolation

Each calculation is associated strictly with the authenticated user. Multi-tenant privacy guarantees User A never accesses User B's records.

Relational DB & Cloudflare Tunnel

PostgreSQL storage with Docker Compose runtime, and zero public open ports via Cloudflare Tunnel outbound routing.

System Architecture & Network Topology

1. User Browser

React 19 SPA (Inertia v3)
Two-way synchronized state
Instant zero-latency preview

2. Cloudflare Edge

SSL/TLS Termination
DDoS Protection & CDN
Custom Domain routing

3. Cloudflare Tunnel

Secure outbound tunnel
Zero exposed public ports
No public IP required

4. Nginx Reverse Proxy

Serves static assets
FastCGI proxy to PHP-FPM
Security headers enforced

5. Laravel 12 Backend

PHP 8.4 runtime
CalculationService
Session & Auth API

6. PostgreSQL DB

Relational schema
Indexed user_id calculations
Persistent named volume

Mathematical Business Logic (AdForecast Pro)

Verified Formulas Matching UI Mockups (ui1.png & ui2.png)

Results Count

$$\text{Results} = \text{Ad Spend} / \text{CPR}$$

Example: $1.5\text{M} / 235\text{k} = 6.38 \rightarrow 6 \text{ results}$ | $5\text{M} / 100\text{k} = 50 \text{ results}$

Revenue

$$\text{Revenue} = \text{Results} * \text{Average Order Value (AOV)}$$

Example: $6.3829 * 10\text{k} = \text{Rp } 63.830$ | $50 * 500\text{k} = \text{Rp } 25.000.000$

Net Profit

$$\text{Profit} = \text{Revenue} - \text{Monthly Ad Spend}$$

Example: $63.830 - 1.5\text{M} = -\text{Rp } 1.436.170 \text{ (Loss)}$ | $25\text{M} - 5\text{M} = +\text{Rp } 20.000.000 \text{ (Profit)}$

ROI %

$$\text{ROI} = ((\text{Revenue} - \text{Spend}) / \text{Spend}) * 100\%$$

Example: $(-1.43\text{M} / 1.5\text{M}) * 100 = -95.7\%$ | $(20\text{M} / 5\text{M}) * 100 = +400.0\%$

CPR Target

$$\text{CPR Target} = 30\% * \text{Product Price}$$

Example: $30\% * 50\text{k} = \text{Rp } 15.000$ | $30\% * 500\text{k} = \text{Rp } 150.000$

Margin per Result

$$\text{Margin / Result} = \text{AOV} - \text{CPR}$$

Example: $10\text{k} - 235\text{k} = -\text{Rp } 225.000$ | $500\text{k} - 100\text{k} = +\text{Rp } 400.000$

Authentication & Multi-Tenant Security

Server-Enforced User Scoping

The backend never trusts a user_id submitted in the request body. All calculations are persisted with `$request->user()->id` and queried exclusively through `$request->user()->calculations()`.

Bcrypt Password Hashing

Passwords hashed with 12 rounds of bcrypt. Plaintext passwords are never logged, stored, or returned in any API responses.

Safe Session Management

Full CSRF protection, session regeneration on login/logout, and secure HTTP-only cookies preventing XSS token exfiltration.

Automated Authorization Tests

Dedicated Pest test suite `CalculationAuthorizationTest` proves that User A cannot read, list, update, or delete User B's calculation records.

Automated Test Suite & Code Quality

94 Tests

335 Assertions

100% Pass Rate

CalculationServiceTest (Unit)

Validates exact decimal precision for mockups 1 & 2, zero CPR division guards, and zero spend boundary handling.

CalculationApiTest (Feature)

Validates REST endpoints POST /api/calculations, store validation ranges, and user isolation barriers.

AuthApiTest (Feature)

Validates registration, email uniqueness, password hashing, session regeneration, and logout invalidation.

Pint & TypeScript Typecheck

Formatted with Laravel Pint (PSR-12/agent standard) and strict TypeScript compiler check (tsc --noEmit).

Docker Compose & Cloudflare Tunnel Deployment

4 Container Production Runtime (docker-compose.prod.yml)

app (PHP 8.4-FPM)

Alpine-based PHP 8.4 with pdo_pgsql, opcache, and optimized autoloading. Runs API & CalculationService.

web (Nginx:Alpine)

Serves pre-compiled Vite React SPA assets and forwards dynamic PHP requests to app:9000.

db (PostgreSQL 16)

Dedicated database container with isolated internal Docker bridge network and persistent postgres_prod_data volume.

cloudflared

Official Cloudflare Tunnel daemon establishing secure outbound connection without exposing any inbound ports.

Project Deliverables Checklist

- Complete clean Git repository with atomic, descriptive commits
- BUSINESS_LOGIC.md documenting inputs, outputs, formula citations, and edge cases
- Comprehensive Pest unit and feature tests covering authentication, formulas, and data isolation
- Production Docker Compose (Nginx, PHP 8.4, PostgreSQL, Cloudflare Tunnel)
- Presentation Deck PDF (presentation.pdf) with architectural & business logic breakdown

Transparent AI Copilot Usage

AI was utilized as an intelligent pair-programmer copilot to rapidly research mathematical marketing benchmarks, draft initial boilerplate schemas, and write regression test suites. Every formula, UI synchronization constraint, database index, and security boundary was systematically analyzed, manually verified against the visual reference mockups, and proved via automated testing.